Tries & Strings — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

409. Implement TRIE | INSERT | SEARCH | STARTSWITH

Understanding the Problem NEW

A Trie (pronounced 'try,' from 'retrieval') is a tree specifically designed for storing strings, where each node represents one character, and following a path from the root spells out a prefix — words that share a common prefix share the same path down the tree. Implement `insert` (add a word), `search` (check if an exact word exists), and `startsWith` (check if any word has this prefix).

Brute Force Approach NEW

Without a Trie, you could store all words in a hashset — `search` would be O(word length) (fine), but `startsWith` would require scanning *every* stored word to check if any of them start with the given prefix — O(number of words × word length), since a plain hashset has no concept of 'shared prefixes.'

Why We Optimize NEW

A Trie's whole design directly targets this weakness: since words sharing a prefix share the same initial path in the tree, checking `startsWith` just means **walking down the tree following the prefix's characters** — if you can walk the whole prefix without hitting a missing child, the prefix exists, in O(prefix length) regardless of how many words are stored. `insert` and `search` are similarly O(word length) — walking down, creating nodes as needed (insert) or checking they all exist and the final node is marked 'end of word' (search). This is the entire reason Tries exist: they turn prefix-related queries from 'expensive because you must consider every stored word' into 'cheap because shared prefixes are pre-organized by the tree's structure itself.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Trie (Prefix Tree) with insert, search, and startsWith operations.

Visual Diagram

```
Trie Structure:
                    root
```

```
                /   |  \
              a     b   c
            /       |     \
          p         o      a
        /           |        \
      p             b        t
    [apple] [bob]          [cat]

    Insert "apple":
    root → a → p → p → l → e
    Mark end at 'e'

    Search "app":
    root → a → p → p → not end → false

    StartsWith "app":
    root → a → p → p → exists → true
```

Solution Approach

Concept: Each node has children array and end marker.

Algorithm

```
Algorithm TRIE:
           class Node:
               children = Node[26]
               isEnd = false

           insert(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       node.children[index] = new Node()
                   node = node.children[index]
               node.isEnd = true

           search(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       Return false

                  node = node.children[index]
              Return node.isEnd

          startsWith(prefix):
              node = root
              For char in prefix:
```

```
                index = char - 'a'
                If node.children[index] == null:
                    Return false
                node = node.children[index]
            Return true
```

Java Implementation

```
class Trie {
          private class Node {
              Node[] children;
              boolean isEnd;

              Node() {
                  children = new Node[26];
                  isEnd = false;
              }
          }

          private Node root;

          public Trie() {
              root = new Node();
          }

          public void insert(String word) {
              Node node = root;
              for (char c : word.toCharArray()) {
                  int index = c - 'a';
                  if (node.children[index] == null) {
                      node.children[index] = new Node();
                  }
                  node = node.children[index];
              }
              node.isEnd = true;
          }

          public boolean search(String word) {
              Node node = root;
              for (char c : word.toCharArray()) {
                  int index = c - 'a';
                  if (node.children[index] == null) {
                      return false;
                  }
                  node = node.children[index];
              }
              return node.isEnd;
          }

          public boolean startsWith(String prefix) {
              Node node = root;
              for (char c : prefix.toCharArray()) {
```

```
                int index = c - 'a';
                if (node.children[index] == null) {
                    return false;
                }
                node = node.children[index];
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(L) | Space Complexity: O(26 * nodes)

410. Implement Trie - 2 (Prefix Tree)

Understanding the Problem NEW

Extend the basic Trie to also support counting: how many words currently stored have a given prefix, and how many equal a given word exactly (useful for a Trie that supports duplicates and deletion).

Brute Force Approach NEW

Without extra bookkeeping in the Trie nodes, you'd have to fall back to scanning every stored word to count prefix or exact-word matches — the same O(words × length) weakness a plain Trie was built to avoid.

Why We Optimize NEW

Store two small counters at **every node**: `countPrefix` (how many words pass through this node) and `countEndsHere` (how many words end exactly at this node) — both updated incrementally during `insert` (incrementing along the path) and `delete` (decrementing along the path, removing nodes whose counts drop to zero). This keeps prefix-count and word-count queries at O(word/prefix length), same as basic search, by maintaining running counts as extra state on nodes you're already visiting during insert/search anyway — no need to ever scan the full word list.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Extended Trie with count of words and prefixes.

Visual Diagram

```
Insert "apple":
       root → a → p → p → l → e
       Counts: each node prefix count++, end at 'e'

       Insert "app":
       root → a → p → p
       Counts: prefix count++, end at 'p'
```

```
countWordsEqualTo("app"): 1
countWordsStartingWith("app"): 2
```

Solution Approach

Concept: Maintain prefix count and end count.

Algorithm

```
Algorithm TRIE_2:
           class Node:
               children = Node[26]
               prefixCount = 0
               endCount = 0

           insert(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       node.children[index] = new Node()
                   node = node.children[index]
                   node.prefixCount++
               node.endCount++

           countWordsEqualTo(word):
               node = root
               For char in word:
                   index = char - 'a'
                   If node.children[index] == null:
                       Return 0
                   node = node.children[index]
               Return node.endCount

           countWordsStartingWith(prefix):
               node = root
               For char in prefix:
                   index = char - 'a'
                   If node.children[index] == null:
                       Return 0
                   node = node.children[index]
               Return node.prefixCount

           erase(word):
               // Decrement counts along path
```

Java Implementation

```
class Trie2 {
         private class Node {
```

```
        Node[] children;
        int prefixCount;
        int endCount;

        Node() {
            children = new Node[26];
            prefixCount = 0;
            endCount = 0;
        }
    }

    private Node root;

    public Trie2() {
        root = new Node();
    }

    public void insert(String word) {
        Node node = root;
        for (char c : word.toCharArray()) {
            int index = c - 'a';
            if (node.children[index] == null) {
                node.children[index] = new Node();
            }
            node = node.children[index];
            node.prefixCount++;
        }
        node.endCount++;
    }

    public int countWordsEqualTo(String word) {
        Node node = root;
        for (char c : word.toCharArray()) {
            int index = c - 'a';
            if (node.children[index] == null) {
                return 0;
            }
            node = node.children[index];
        }
        return node.endCount;
    }

    public int countWordsStartingWith(String prefix) {
        Node node = root;
        for (char c : prefix.toCharArray()) {
            int index = c - 'a';
            if (node.children[index] == null) {
                return 0;
            }
            node = node.children[index];
        }
        return node.prefixCount;
    }
```

```
        public void erase(String word) {
            if (countWordsEqualTo(word) == 0) return;
            eraseHelper(root, word, 0);
        }

        private boolean eraseHelper(Node node, String word, int index) {
            if (index == word.length()) {
                node.endCount--;
                return true;
            }

            int idx = word.charAt(index) - 'a';
            if (eraseHelper(node.children[idx], word, index + 1)) {
                node.children[idx].prefixCount--;
                if (node.children[idx].prefixCount == 0) {
                    node.children[idx] = null;
                }
                return true;
            }
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(L) | Space Complexity: O(26 * nodes)

411. Longest String with All Prefixes

Understanding the Problem NEW

Given a list of words, find the longest word such that every one of its prefixes (of every length) is also present somewhere in the list.

Brute Force Approach NEW

For every word, check whether every one of its prefixes exists in the word list — using a hashset, each prefix check is O(1) average, but checking all prefixes of all words gives O(total characters × average word length) if not organized well, and doesn't easily let you build up longer valid words incrementally.

Why We Optimize NEW

Insert every word into a **Trie**, marking end-of-word nodes; then do a DFS/BFS traversal of the Trie, but only continuing into a child if that child is itself marked as an end-of-word (ensuring the prefix built so far is a genuine complete word from the list, not just a partial path) — track the longest such path found. This directly exploits the Trie's prefix-sharing structure: 'is every prefix also a complete word' becomes 'does every step down this path pass through an end-of-word marker,' checkable during a single traversal rather than independently verifying every prefix of every word against a separate hashset.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest string where all prefixes exist.

Visual Diagram

```
Words: ["a", "banana", "app", "appl", "ap", "apply", "apple"]
       Longest: "apple" (all prefixes: a, ap, app, appl, apple exist)

       Using Trie:
       Insert all words, mark end
       DFS from root to find longest word
```

Solution Approach

Concept: Trie with DFS traversal.

Algorithm

```
Algorithm LONGEST_WORD(words):
           Insert all words into Trie
           DFS from root:
               If node.isEnd:
                   For each child:
                       DFS(child)
                   Update max length
           Return longest word
```

Java Implementation

```
class Solution {
           private String longest = "";

            public String longestWord(String[] words) {
                Trie trie = new Trie();
                for (String word : words) {
                    trie.insert(word);
                }
                dfs(trie.root, new StringBuilder());
                return longest;
            }

            private void dfs(Trie.Node node, StringBuilder sb) {
                for (int i = 0; i < 26; i++) {
                    Trie.Node child = node.children[i];
                    if (child != null && child.isEnd) {
                        sb.append((char)('a' + i));
                        if (sb.length() > longest.length()) {
                            longest = sb.toString();
                        }
                        dfs(child, sb);
                        sb.deleteCharAt(sb.length() - 1);
                    }
                }
            }
      }
```

Time & Space Complexity

Time Complexity: O(26 * L) | Space Complexity: O(26 * nodes)

412. Number of Distinct Substrings in a String

Understanding the Problem NEW

Count the number of distinct (non-repeating) substrings of a given string.

Brute Force Approach NEW

Generate every possible substring ($O(n^2)$ of them) and use a hashset to deduplicate — $O(n^2)$ substrings, each potentially $O(n)$ to hash/compare, giving $O(n^3)$ in the worst case.

Why We Optimize NEW

Insert every suffix of the string into a **Trie** character by character — since every substring is a prefix of some suffix, this Trie ends up containing exactly one path for every distinct substring. Counting distinct substrings is then simply **counting the number of nodes created** in the Trie (each new node represents one newly-discovered distinct substring that didn't exist as a prefix before) — achieved in $O(n^2)$ total (n suffixes, each up to length n), better than the $O(n^3)$ hashset approach, by letting the Trie's structure itself naturally deduplicate shared prefixes as they're inserted, rather than explicitly generating and hashing every substring separately.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count distinct substrings using Trie.

Visual Diagram

```
s = "ababa"
       Distinct substrings: 9
       "a", "b", "ab", "ba", "aba", "bab", "abab", "baba", "ababa"

       Insert all suffixes:
       "ababa" → a,b,a,b,a
       "baba" → b,a,b,a
       "aba" → a,b,a
       "ba" → b,a
       "a" → a
```

```
       Count distinct nodes in Trie = 9
```

Solution Approach

Concept: Insert suffixes into Trie.

Algorithm

```
Algorithm DISTINCT_SUBSTRINGS(s):
           trie = new Trie()
           count = 0

            For i = 0 to n-1:
                node = root
                For j = i to n-1:
                    index = s[j] - 'a'
                    If node.children[index] == null:
                        node.children[index] = new Node()
                        count++
                    node = node.children[index]

            Return count
```

Java Implementation

```
class Solution {
           public int countDistinctSubstrings(String s) {
               Node root = new Node();
               int count = 0;
               int n = s.length();

                 for (int i = 0; i < n; i++) {
                     Node node = root;
                     for (int j = i; j < n; j++) {
                         int idx = s.charAt(j) - 'a';
                         if (node.children[idx] == null) {
                             node.children[idx] = new Node();
                             count++;
                         }
                         node = node.children[idx];
                     }
                 }
                 return count + 1; // +1 for empty substring
            }

           private class Node {
               Node[] children = new Node[26];
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

413. Bit PreRequisites for TRIE Problems

Understanding the Problem NEW

Before tackling XOR-based Trie problems, this section reviews the bit-manipulation prerequisites: representing numbers in fixed-width binary and understanding how a Trie can be built over the *bits* of numbers (rather than characters of a string) to answer XOR-maximization queries efficiently.

Brute Force Approach NEW

Nothing to contrast — this is preparatory material connecting back to the Bit Manipulation chapter's XOR properties (problems 151-168), needed before the following problems make sense.

Why We Optimize NEW

The key idea being set up here: just as a string-based Trie organizes words by shared character-prefixes, a **bit-Trie** organizes numbers by shared binary-prefixes (processing each number's bits from most-significant to least-significant) — and because XOR-ing two numbers gives the largest result when their bits differ as much as possible, a bit-Trie lets you greedily search, at each bit position, for 'the opposite bit if it's available' — turning an $O(n^2)$ all-pairs XOR check into something much faster, which is exactly what problems 414-415 build on.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding bit manipulation for Trie.

Visual Diagram

```
Binary numbers: [3, 10, 5, 25, 2, 8]
       3 = 00011
       10 = 01010
       5 = 00101
       25 = 11001
       2 = 00010
       8 = 01000

       Bit Trie:
                     root
```

```
                  /      \
                 0        1
                / \      / \
               0   1 1      0
               ...
```

Solution Approach

Concept: Build Trie with bits 0 and 1.

Java Implementation

```
class BitTrie {
           private class Node {
               Node[] children = new Node[2];
           }

            private Node root;

            BitTrie() {
                root = new Node();
            }

            void insert(int num) {
                Node node = root;
                for (int i = 31; i >= 0; i--) {
                    int bit = (num >> i) & 1;
                    if (node.children[bit] == null) {
                        node.children[bit] = new Node();
                    }
                    node = node.children[bit];
                }
            }

            int maxXor(int num) {
                Node node = root;
                int maxXor = 0;
                for (int i = 31; i >= 0; i--) {
                    int bit = (num >> i) & 1;
                    int opposite = 1 - bit;
                    if (node.children[opposite] != null) {
                        maxXor |= (1 << i);
                        node = node.children[opposite];
                    } else {
                        node = node.children[bit];

            }
        }
        return maxXor;
```

```
    }
}
```

414. Maximum XOR of Two Numbers in an Array

Understanding the Problem NEW

Given an array of numbers, find the maximum XOR value achievable between any two of them.

Brute Force Approach NEW

Check every pair of numbers and compute their XOR directly — O(n²) pairs.

Why We Optimize NEW

Insert every number into a **bit-Trie** (each number's binary representation, most-significant bit first, forming one root-to-leaf path). Then, for each number, walk the Trie trying to **greedily choose the opposite bit** at every position (since XOR-ing differing bits gives 1s, maximizing the result) — if the opposite bit's branch exists in the Trie, take it (this contributes a 1 to the result at this position); otherwise, you're forced to take the same-bit branch (contributing a 0). Doing this for every number, each in O(bit-width) time using the pre-built Trie, achieves O(n × bit-width) total instead of O(n²) — directly reusing the greedy 'always prefer the branch that helps most' idea, applied to bits instead of characters.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum XOR of any two numbers.

Visual Diagram

```
Array: [3, 10, 5, 25, 2, 8]
       Max XOR: 28 (25 ^ 2 = 11001 ^ 00010 = 11011 = 27)
       Actually max is 28 (25 ^ 5 = 28)

       Insert all numbers into bit Trie
       For each num, find max XOR with existing nums
```

Solution Approach

Concept: Bit Trie with XOR query.

Algorithm

```
Algorithm MAX_XOR(nums):
           root = new BitTrie()
           maxXor = 0

           For num in nums:
               root.insert(num)
               maxXor = max(maxXor, root.maxXor(num))

           Return maxXor
```

Java Implementation

```
class Solution {
           public int findMaximumXOR(int[] nums) {
               BitTrie trie = new BitTrie();
               int maxXor = 0;

                 for (int num : nums) {
                     trie.insert(num);
                     maxXor = Math.max(maxXor, trie.maxXor(num));
                 }
                 return maxXor;
           }

           private class BitTrie {
               private class Node {
                   Node[] children = new Node[2];
               }

                 private Node root;

                 BitTrie() {
                     root = new Node();
                 }

                 void insert(int num) {
                     Node node = root;
                     for (int i = 31; i >= 0; i--) {
                         int bit = (num >> i) & 1;
                         if (node.children[bit] == null) {
                             node.children[bit] = new Node();
                         }

                           node = node.children[bit];
                       }
               }

               int maxXor(int num) {
                   Node node = root;
```

```
                int maxXor = 0;
                for (int i = 31; i >= 0; i--) {
                    int bit = (num >> i) & 1;
                    int opposite = 1 - bit;
                    if (node.children[opposite] != null) {
                        maxXor |= (1 << i);
                        node = node.children[opposite];
                    } else {
                        node = node.children[bit];
                    }
                }
                return maxXor;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * 32) | Space Complexity: O(n * 32)

415. Maximum XOR With an Element From Array

Understanding the Problem NEW

A variant of Maximum XOR: given several queries, each asking for the maximum XOR achievable between a given value x and any array element that is ≤ some limit m, answer all queries efficiently.

Brute Force Approach NEW

For each query, filter the array down to elements ≤ m, then check every remaining pair with x for XOR — O(queries × n) in the worst case.

Why We Optimize NEW

Sort the array, and **sort the queries by their limit m**; then process both in increasing order together, inserting array elements into the bit-Trie (414) only once they're ≤ the current query's limit (since queries are processed in increasing-m order, elements only ever need to be added, never removed) — then answer each query using the same greedy bit-Trie walk as problem 414, but only among elements already inserted (i.e., only those ≤ m). This 'sort queries and process incrementally, only ever adding to the Trie' technique avoids rebuilding or re-filtering the Trie per query, achieving roughly O((n + queries) log n) total instead of O(queries × n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

For each query (xi, mi), find max xi XOR y where y ≤ mi.

Visual Diagram

```
nums = [0,1,2,3,4]
       queries = [[3,1],[1,3],[5,6]]

       Query [3,1]: nums ≤ 1: [0,1], max xor = 3^0=3
       Query [1,3]: nums ≤ 3: [0,1,2,3], max xor = 1^2=3
       Query [5,6]: nums ≤ 6: all, max xor = 5^2=7
```

Solution Approach

Concept: Sort queries and nums, offline processing.

Algorithm

```
Algorithm MAX_XOR_QUERIES(nums, queries):
           Sort nums
           Sort queries by mi
           Trie trie = new Trie()
           result = new int[queries.length]
           index = 0

           For query in queries:
               While index < nums.length && nums[index] <= mi:
                   trie.insert(nums[index])
                   index++
               If trie.isEmpty(): result[i] = -1
               Else: result[i] = trie.maxXor(xi)

           Return result
```

Java Implementation

```
class Solution {
           public int[] maximizeXor(int[] nums, int[][] queries) {
               Arrays.sort(nums);
               int n = queries.length;
               int[][] q = new int[n][3];
               for (int i = 0; i < n; i++) {
                   q[i][0] = queries[i][0];
                   q[i][1] = queries[i][1];
                   q[i][2] = i;
               }
               Arrays.sort(q, (a, b) -> a[1] - b[1]);

                 BitTrie trie = new BitTrie();
                 int[] result = new int[n];
                 int index = 0;

                 for (int[] query : q) {
                     int xi = query[0], mi = query[1], qi = query[2];
                     while (index < nums.length && nums[index] <= mi) {
                         trie.insert(nums[index]);
                         index++;
                     }
                     result[qi] = index == 0 ? -1 : trie.maxXor(xi);
                 }

               return result;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O((n+m) * 32) | Space Complexity: O(n * 32)

416. Minimum Number of Bracket Reversals

Understanding the Problem NEW

Given a string of brackets that might not be balanced (but has an even length), find the minimum number of bracket reversals (flipping '(' to ')' or vice versa) needed to make it valid.

Brute Force Approach NEW

Try every possible combination of reversals and check validity — exponential without structure.

Why We Optimize NEW

After removing every already-matched pair (using a stack, similar to the Balanced Parentheses check from the Stack/Queue chapter, problem 175), what remains is some number of unmatched `)` characters followed by unmatched `(` characters (a specific, simple leftover pattern once matched pairs are stripped away) — counting the leftover unmatched `)`s as `m` and unmatched `(`s as `n`, the answer is `ceil(m/2) + ceil(n/2)` (each mismatched pair of the *same* type needs exactly one reversal to fix, and any single leftover odd one needs two reversals since it must flip type entirely). This reduces the reversal-counting problem to a straightforward stack-based reduction plus simple counting, reusing the same stack-matching idea from the Stack/Queue chapter rather than exploring reversal combinations directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum reversals to make valid parentheses.

Visual Diagram

```
s = "}{{}}{{{"
      Invalid, min reversals = 2

      Count unmatched:
      Step 1: remove valid pairs
```

```
      remaining = "}{{"
      Step 2: count = 2, result = ceil(count/2)
```

Solution Approach

Concept: Count unmatched brackets.

Algorithm

```
Algorithm MIN_REVERSALS(s):
          If n % 2 != 0: Return -1
          left = 0, right = 0

          For char in s:
              If char == '{':
                  left++
              Else:
                  If left == 0:
                      right++
                  Else:
                      left--

          Return (left + 1) / 2 + (right + 1) / 2
```

Java Implementation

```
class Solution {
          public int minReversals(String s) {
              int n = s.length();
              if (n % 2 != 0) return -1;

                 int open = 0, close = 0;
                 for (char c : s.toCharArray()) {
                     if (c == '{') {
                         open++;
                     } else {
                         if (open == 0) {
                              close++;
                         } else {
                              open--;
                         }
                     }
                 }
                 return (open + 1) / 2 + (close + 1) / 2;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

417. Count and Say

Understanding the Problem NEW

Generate the n-th term of the 'count-and-say' sequence, where each term describes the previous term by counting consecutive runs of the same digit (e.g. '1' → '11' [one 1] → '21' [two 1s] → '1211' [one 2, one 1]).

Brute Force Approach NEW

There's no faster alternative — you genuinely need to build each term from the previous one in sequence, since each term is defined purely in terms of describing its predecessor.

Why We Optimize NEW

Simply simulate the definition directly: for each term, scan through the previous term's digits, counting consecutive runs, and build the next term as (count, digit) pairs concatenated together — O(length of term) per step, and since each term's length can roughly grow with each step, the overall cost through n terms depends on how quickly the sequence grows, but there's no cleverer shortcut than direct simulation here, since the sequence has no simpler closed-form pattern to exploit.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate nth count-and-say sequence.

Visual Diagram

```
1: "1"
       2: "11" (one 1)
       3: "21" (two 1s)
       4: "1211" (one 2, one 1)
       5: "111221" (one 1, one 2, two 1s)

       Build next from current:
       Current: "1211"
       Count runs:
```

```
       1→1, 2→1, 11→2
       Next: "111221"
```

Solution Approach

Concept: Run-length encoding.

Algorithm

```
Algorithm COUNT_AND_SAY(n):
           result = "1"
           For i = 2 to n:
               next = ""
               count = 1
               For j = 1 to result.length():
                   If j == result.length() || result[j] != result[j-1]:
                        next += count + result[j-1]
                        count = 1
                   Else:
                        count++
               result = next
           Return result
```

Java Implementation

```
class Solution {
           public String countAndSay(int n) {
               String result = "1";
               for (int i = 2; i <= n; i++) {
                   StringBuilder next = new StringBuilder();
                   int count = 1;
                   for (int j = 1; j < result.length(); j++) {
                        if (result.charAt(j) != result.charAt(j-1)) {
                            next.append(count).append(result.charAt(j-1));
                            count = 1;
                        } else {
                            count++;
                        }
                   }
                   next.append(count).append(result.charAt(result.length()-1));
                   result = next.toString();
               }
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n)

418-419. Rabin Karp Algorithm

Understanding the Problem NEW

Search for a pattern within a text string — the classic pattern-matching problem, here solved with the **Rabin-Karp** algorithm, which uses hashing to speed up comparisons.

Brute Force Approach NEW

The naive approach checks the pattern against every possible starting position in the text, comparing character by character at each position — O(n×m) in the worst case (n = text length, m = pattern length), since a near-match can require almost a full pattern-length comparison before failing.

Why We Optimize NEW

Compute a **rolling hash** of the pattern and of every same-length window of the text — critically, the hash of the *next* window can be computed from the *current* window's hash in O(1) (by removing the outgoing character's contribution and adding the incoming character's, using modular arithmetic), rather than rehashing the whole window from scratch. Only when a window's hash matches the pattern's hash do you do a full character-by-character verification (to rule out hash collisions) — in practice, this achieves close to O(n+m) average time, since most non-matching windows are ruled out by a quick O(1) hash comparison instead of a full O(m) character comparison, though a poor hash function (with many collisions) can degrade this back toward O(n×m) in the worst case.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Pattern matching using rolling hash.

Visual Diagram

```
Text: "ababcab", Pattern: "abc"
       Hash("abc") = 123
       Rolling hash:
       "aba" → remove 'a', add 'c' → "abc"
       When hash matches, verify actual string
```

Solution Approach

Concept: Rolling hash with modulo.

Algorithm

```
Algorithm RABIN_KARP(text, pattern):
           n = text.length, m = pattern.length
           if m > n: return []
           d = 256, q = 101 (prime)

           // Compute hash
           p = 0, t = 0, h = d^(m-1) % q
           For i = 0 to m-1:
               p = (p*d + pattern[i]) % q
               t = (t*d + text[i]) % q

           For i = 0 to n-m:
               If p == t:
                   If text.substring(i, i+m) == pattern:
                       return i
               If i < n-m:
                   t = (d*(t - text[i]*h) + text[i+m]) % q
                   If t < 0: t += q
           Return -1
```

Java Implementation

```
class Solution {
           public int rabinKarp(String text, String pattern) {
               int n = text.length(), m = pattern.length();
               if (m > n) return -1;

                 int d = 256, q = 101;
                 int h = 1;
                 for (int i = 0; i < m-1; i++) {
                     h = (h * d) % q;
                 }

                 int p = 0, t = 0;
                 for (int i = 0; i < m; i++) {
                     p = (p * d + pattern.charAt(i)) % q;
                     t = (t * d + text.charAt(i)) % q;
                 }

                 for (int i = 0; i <= n - m; i++) {
                     if (p == t) {
                         if (text.substring(i, i+m).equals(pattern)) {
                             return i;

                      }
```

```
                }
                if (i < n - m) {
                    t = (d * (t - text.charAt(i) * h) + text.charAt(i+m)) % q;
                    if (t < 0) t += q;
                }
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n+m) average | Space Complexity: O(1)

420. Z-Function

Understanding the Problem NEW

Compute the Z-array for a string — where `Z[i]` is the length of the longest substring starting at position i that matches a prefix of the string itself. This array is a powerful building block for various pattern-matching tasks.

Brute Force Approach NEW

Compute each `Z[i]` independently by directly comparing characters starting at position i against the string's prefix, character by character — O(n) per position, O(n²) overall, since many of these comparisons redundantly re-examine characters already compared for a previous position.

Why We Optimize NEW

Maintain a **'Z-box'** — the rightmost-reaching interval [L, R] found so far where a prefix-match is known to extend — and when computing a new position, if it falls within an existing Z-box, you can initialize its comparison using already-known information from an earlier corresponding position (since the Z-box guarantees that segment already matches the prefix), only extending the comparison further if needed, rather than starting completely from scratch. This 'reuse previously discovered matching information via the current Z-box' technique computes the entire Z-array in O(n) total, instead of the O(n²) that comes from treating every position as an independent, from-scratch comparison.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Compute Z-array for pattern matching.

Visual Diagram

```
s = "aabcaabx"
       Z-array:
       [0,1,0,0,3,1,0,0]

       Z[i] = longest prefix of s starting at i
       At i=4: "aabx" matches prefix "aab" → 3
```

Solution Approach

Concept: Z-algorithm with [L,R] window.

Algorithm

```
Algorithm Z_FUNCTION(s):
           n = s.length()
           z = new int[n]
           l = 0, r = 0

           For i = 1 to n-1:
               If i <= r:
                   z[i] = min(r - i + 1, z[i - l])
               While i + z[i] < n && s[z[i]] == s[i + z[i]]:
                   z[i]++
               If i + z[i] - 1 > r:
                   l = i
                   r = i + z[i] - 1

           Return z
```

Java Implementation

```
class Solution {
           public int[] zFunction(String s) {
               int n = s.length();
               int[] z = new int[n];
               int l = 0, r = 0;

                 for (int i = 1; i < n; i++) {
                     if (i <= r) {
                         z[i] = Math.min(r - i + 1, z[i - l]);
                     }
                     while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) {
                         z[i]++;
                     }
                     if (i + z[i] - 1 > r) {
                         l = i;
                         r = i + z[i] - 1;
                     }
                 }
                 return z;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

421. KMP Algorithm / LPS (pi) Array

Understanding the Problem NEW

The Knuth-Morris-Pratt (KMP) algorithm for pattern matching, built around computing the LPS (Longest Proper Prefix that is also a Suffix) array for the pattern, used to search a pattern within a text efficiently.

Brute Force Approach NEW

The naive approach (same as Rabin-Karp's baseline) checks the pattern against every text position independently, and on a mismatch, restarts the comparison from the very next text position and the very start of the pattern — O(n×m) worst case, since it discards all the partial-match information gained before the mismatch.

Why We Optimize NEW

Precompute the pattern's **LPS array** (for each prefix of the pattern, the length of its longest proper prefix that's also a suffix of that same prefix) — this captures exactly how much of a partial match can be 'reused' when a mismatch occurs, since the pattern itself is compared against itself to find repeated internal structure. During the actual text search, on a mismatch, instead of restarting the pattern from position 0, you jump the pattern pointer back to `LPS[previous position]` — skipping past comparisons that the LPS array guarantees would fail anyway, since that structure was already confirmed not to match. This achieves O(n+m) total (O(m) to build the LPS array, O(n) for the search), a genuine improvement over the O(n×m) naive approach, by never re-examining text characters that have already been compared once — a different, generally faster-worst-case alternative to Rabin-Karp's hash-based approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Pattern matching using KMP.

Visual Diagram

```
Pattern: "ababc"
       LPS: [0,0,1,2,0]
```

```
       LPS[i] = longest proper prefix which is also suffix
       "abab" → prefix "ab", suffix "ab" → 2

       KMP search:
       Text: "abababc"
       Pattern: "ababc"
       Match found at index 2
```

Solution Approach

Concept: LPS array for failure function.

Algorithm

```
Algorithm KMP(text, pattern):
           lps = computeLPS(pattern)
           i = 0, j = 0

            While i < text.length:
                If text[i] == pattern[j]:
                    i++; j++
                    If j == pattern.length:
                        return i - j
                Else If j > 0:
                    j = lps[j-1]
                Else:
                    i++
            Return -1
```

Java Implementation

```
class Solution {
           public int kmpSearch(String text, String pattern) {
               int[] lps = computeLPS(pattern);
               int i = 0, j = 0;

                 while (i < text.length()) {
                     if (text.charAt(i) == pattern.charAt(j)) {
                         i++;
                         j++;
                         if (j == pattern.length()) {
                              return i - j;
                         }
                     } else if (j > 0) {
                         j = lps[j-1];
                     } else {
                         i++;
                     }
                  }
```

```
                return -1;
            }
            private int[] computeLPS(String pattern) {

              int[] lps = new int[pattern.length()];
              int len = 0;
              int i = 1;

              while (i < pattern.length()) {
                  if (pattern.charAt(i) == pattern.charAt(len)) {
                      len++;
                      lps[i] = len;
                      i++;
                  } else if (len > 0) {
                      len = lps[len - 1];
                  } else {
                      lps[i] = 0;
                      i++;
                  }
              }
              return lps;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n+m) | Space Complexity: O(m)

422. Shortest Palindrome

Understanding the Problem NEW

Given a string, find the shortest palindrome you can create by adding characters only to the *front* of the string.

Brute Force Approach NEW

Try every possible prefix to add and check if the resulting string is a palindrome — O(n²) checks (n possible insertion points, each requiring an O(n) palindrome check), or worse if not done carefully.

Why We Optimize NEW

The key question is: what's the **longest palindromic prefix** of the string (starting from index 0)? — everything *after* that palindromic prefix needs to have its reverse added to the front. To find this efficiently, construct a new string as `string + separator + reverse(string)` and compute its **KMP LPS array** (421) — the LPS value at the very last position of this combined string tells you exactly how long of a prefix of the original string is also a palindrome (since the LPS captures the longest overlap between the string and its own reverse, arranged this way). This reuses the KMP LPS computation (O(n)) to answer a seemingly different palindrome question, rather than checking every possible prefix insertion directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add characters to front to make palindrome.

Visual Diagram

```
s = "aacecaaa"
       Shortest palindrome: "aaacecaaa"

       Find longest palindrome prefix:
       "aacecaa" is palindrome? "aacecaa" reversed "aacecaa" yes
       Add remaining "aa" to front? Actually "aa"+"aacecaaa" = "aaaacecaaa"

       Using KMP:
```

```
       s + "#" + reverse(s)
       Find LPS of combined string
```

Solution Approach

Concept: KMP LPS on combined string.

Algorithm

```
Algorithm SHORTEST_PALINDROME(s):
           rev = reverse(s)
           combined = s + "#" + rev
           lps = computeLPS(combined)
           toAdd = rev.substring(0, s.length() - lps[lps.length-1])
           Return toAdd + s
```

Java Implementation

```
class Solution {
           public String shortestPalindrome(String s) {
               String rev = new StringBuilder(s).reverse().toString();
               String combined = s + "#" + rev;
               int[] lps = computeLPS(combined);
               int len = lps[lps.length - 1];
               String toAdd = rev.substring(0, s.length() - len);
               return toAdd + s;
           }

           private int[] computeLPS(String s) {
               int[] lps = new int[s.length()];
               int len = 0, i = 1;
               while (i < s.length()) {
                   if (s.charAt(i) == s.charAt(len)) {
                       len++;
                       lps[i] = len;
                       i++;
                   } else if (len > 0) {
                       len = lps[len - 1];
                   } else {
                       i++;
                   }
               }
               return lps;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

423. Longest Happy Prefix

Understanding the Problem NEW

Find the length of the longest 'happy prefix' of a string — a substring that is both a proper prefix (not the whole string) and a proper suffix of the same string.

Brute Force Approach NEW

Check every possible prefix length and see if the same-length suffix matches it — $O(n^2)$ in the worst case (n possible lengths, each requiring an O(n) comparison).

Why We Optimize NEW

This is *exactly* what the KMP **LPS array** (421) computes at its very last position — the LPS array's definition ('longest proper prefix that is also a suffix,' computed for every prefix of the string) directly gives the answer for the whole string at its final index, with no modification needed. This problem exists specifically to make explicit that the LPS array — usually thought of as just an internal building block for KMP's pattern search — directly answers this seemingly separate-sounding question on its own, in the O(n) time it already takes to build the array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Longest prefix which is also suffix.

Visual Diagram

```
s = "ababab"
       Longest happy prefix: "abab"
       LPS array: [0,0,1,2,3,4]
       Result: s.substring(0, 4) = "abab"

       s = "level"
       LPS: [0,0,0,0,1]
       Result: "l"
```

Solution Approach

Concept: KMP LPS array.

Algorithm

```
Algorithm LONGEST_HAPPY_PREFIX(s):
           lps = computeLPS(s)
           len = lps[lps.length - 1]
           Return len == 0 ? "" : s.substring(0, len)
```

Java Implementation

```
class Solution {
           public String longestHappyPrefix(String s) {
               int[] lps = computeLPS(s);
               int len = lps[lps.length - 1];
               return len == 0 ? "" : s.substring(0, len);
           }

            private int[] computeLPS(String s) {
                int[] lps = new int[s.length()];
                int len = 0, i = 1;
                while (i < s.length()) {
                    if (s.charAt(i) == s.charAt(len)) {
                        len++;
                        lps[i] = len;
                        i++;
                    } else if (len > 0) {
                        len = lps[len - 1];
                    } else {
                        i++;
                    }
                }
                return lps;
            }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

424. Count Palindromic Subsequences in Given String

Understanding the Problem NEW

Count the number of palindromic subsequences within a string (not necessarily distinct — every valid selection counts, even if it produces the same characters as another selection).

Brute Force Approach NEW

Try every one of the 2^n subsequences and check which are palindromes — exponential.

Why We Optimize NEW

This connects back to the Dynamic Programming chapter's interval-DP family: define `countPalindromes(left, right)` using inclusion-exclusion — count the palindromic subsequences within `[left+1, right]` plus those within `[left, right-1]`, but subtract the double-counted ones in `[left+1, right-1]` to avoid overcounting, and if the characters at `left` and `right` match, add back in the subsequences formed by wrapping that matching pair around every subsequence (including the empty one) within `[left+1, right-1]`. This is the same 'range as DP state' interval-DP shape from the DP chapter (Matrix Chain Multiplication, Burst Balloons), applied here to a counting question about palindromic subsequences, achieving $O(n^2)$ instead of the exponential 2^n subsequence enumeration — a fitting final problem for the book, since it ties this last chapter's string-matching machinery (Tries, KMP, Z-function) back to the interval-DP technique from the chapter just before it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count palindromic subsequences in string.

Visual Diagram

```
s = "bccb"
       Palindromic subsequences: 7
       "b", "c", "b", "c", "bb", "cc", "bccb"
```

```
dp[i][j] = count of palindromic subsequences in s[i..j]
If s[i] == s[j]:
    dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1
Else:
    dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]
```

Solution Approach

Concept: DP with inclusion-exclusion.

Algorithm

```
Algorithm COUNT_PAL_SUBSEQ(s):
           n = s.length()
           dp = int[n][n]

           For i = 0 to n-1: dp[i][i] = 1

           For len = 2 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   If s[i] == s[j]:
                       dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1
                   Else:
                       dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]

           Return dp[0][n-1]
```

Java Implementation

```
class Solution {
           public int countPalindromicSubsequences(String s) {
               int n = s.length();
               int[][] dp = new int[n][n];

                 for (int i = 0; i < n; i++) {
                     dp[i][i] = 1;
                 }

                 for (int len = 2; len <= n; len++) {
                     for (int i = 0; i <= n - len; i++) {
                         int j = i + len - 1;
                         if (s.charAt(i) == s.charAt(j)) {
                             dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1;
                         } else {
                             dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1];
                         }
                     }
                 }
```

```
                return dp[0][n-1];
            }

        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

Tries & Strings Summary

Problem	Category	Time	Space	Key Technique
Basic Trie	Trie	O(L)	O(26*nodes)	Prefix Tree
Trie 2	Trie	O(L)	O(26*nodes)	Counts tracking
Longest Word	Trie	O(26*L)	O(26*nodes)	DFS traversal
Distinct Substrings	Trie	$O(n^2)$	$O(n^2)$	Suffix insertion
Max XOR	Bit Trie	O(n*32)	O(n*32)	Bit Trie
Max XOR Queries	Bit Trie	O((n+m)*32)	O(n*32)	Offline processing
Min Reversals	Strings	O(n)	O(1)	Unmatched count
Count and Say	Strings	O(2^n)	O(2^n)	Run-length encoding
Rabin Karp	Strings	O(n+m)	O(1)	Rolling hash
Z-Function	Strings	O(n)	O(n)	Z-array
KMP	Strings	O(n+m)	O(m)	LPS array
Shortest Palindrome	Strings	O(n)	O(n)	KMP + reverse
Happy Prefix	Strings	O(n)	O(n)	LPS array
Count Pal Subseq	Strings	$O(n^2)$	$O(n^2)$	Interval DP

— End of enhanced Chapter 14: Tries & Strings — End of Enhanced Beginner Edition —